KU LEUVEN

Verslag geavanceerde computerarchitectuur

— Sessie 3: Warps, thread divergence, and striding —

Daan Vercammen
Klaas Meersman

ELICT: ELECFS
7 maart 2025
Master in industriële wetenschappen
Academiejaar 2024-2025

1.1 Probleemstelling en doel

In dit labo doen we inzichten op over warps, striding en thread divergentie. Dat doen we aan de hand van een inversie van een afbeelding en vervolgens door het toepassen van een Gaussiaanse blur-functie. Door het aantal threads te variëren en striding te implementeren, onderzoeken we hoe GPU-prestaties beïnvloed worden door de organisatie van threads in warps. Ook onderzoeken we thread divergentie door de bewerking van een afbeelding op verschillende threads te organiseren. We proberen de efficiëntie te verbeteren door verschillende technieken toe te passen.

1.2 Methodologie

Eerst vergelijken we uitvoeringstijd van de CPU t.o.v. de GPU bij zowel het inverteren als het toepassen van een Gaussiaanse filter. Vervolgens focussen we ons op de invloed van striding op de uitvoeringstijd. Striding wordt gebruikt wanneer er niet genoeg threads zijn om alle data-elementen parallel te verwerken. Als laatste analyseren we de effecten van thread divergence en proberen we de invloed ervan te minimaliseren door eerst een transformatie van de data uit te voeren en zo de verwerking van de pixels anders te organiseren. Om betrouwbare resultaten te verkrijgen, worden er meerdere runs van elke meting uitgevoerd. De mediaan van deze runs wordt vervolgens berekend om een representatieve waarde te verkrijgen, die minder gevoelig is voor uitschieters dan het gemiddelde.

1.3 Resultaten en observaties

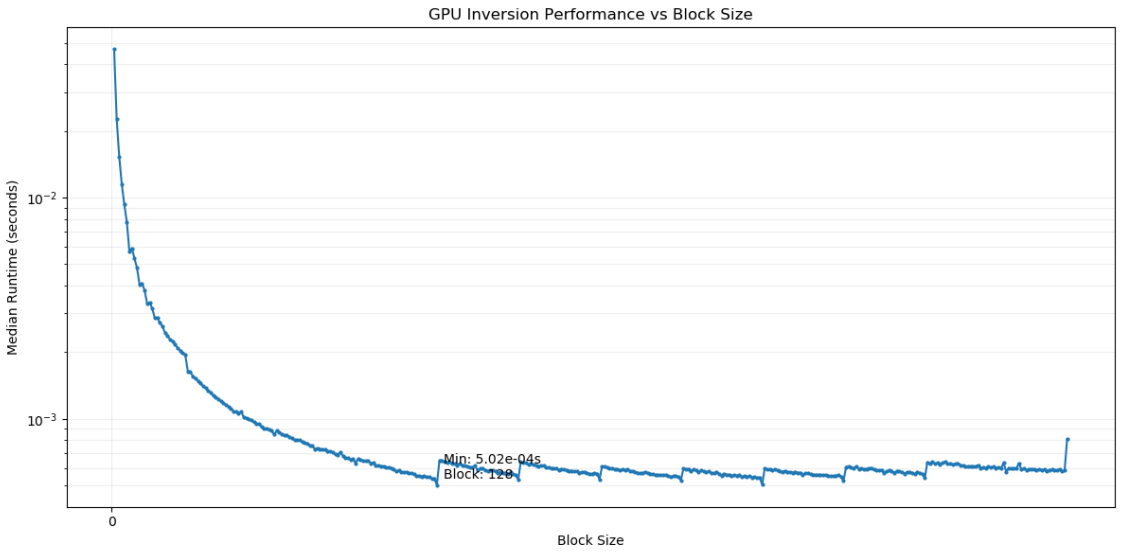
1.3.1 CPU vs. GPU

We nemen de algemene tendens waar dat zowel voor de inversie als voor de inversie met Gaussiaanse blur de GPU performanter is dan de CPU. De Gaussiaanse blur-bewerking vereist meer rekentijd dan de inversie. Dit is te verwachten, aangezien de blur-bewerking vermenigvuldigingen en optellingen omvat, wat complexere berekeningen zijn dan de eenvoudige aftrekking die bij de inversie wordt gebruikt.

1.3.2 Striding

Vervolgens onderzoeken we de invloed van de blocksize op de runtime in de GPU voor een algoritme met striding, `inversionGPU` functie op lijn 132 in appendix A. Striding wordt gebruikt wanneer het aantal beschikbare threads kleiner is dan het aantal elementen dat parallel verwerkt kan worden. In Figuur 1.1 zien we dat de runtime exponentieel afneemt, vervolgens afvlakt,en kleine toenames in uitvoertijd vertoont bij veelvouden van 32 threads.

Deze sprongen zijn te wijten aan de warp-grootte van 32 threads. Hoe dichter de blocksize ligt bij een veelvoud van 32 is, hoe efficiënter de threads worden uitgevoerd. Wanneer de blocksize een veelvoud van 32 overschrijdt, neemt de runtime tijdelijk toe door de inefficiëntie die ontstaat bij de activering van extra warps. Dat kost immers meer tijd dan dat individuele threads binnen een warp opleveren.

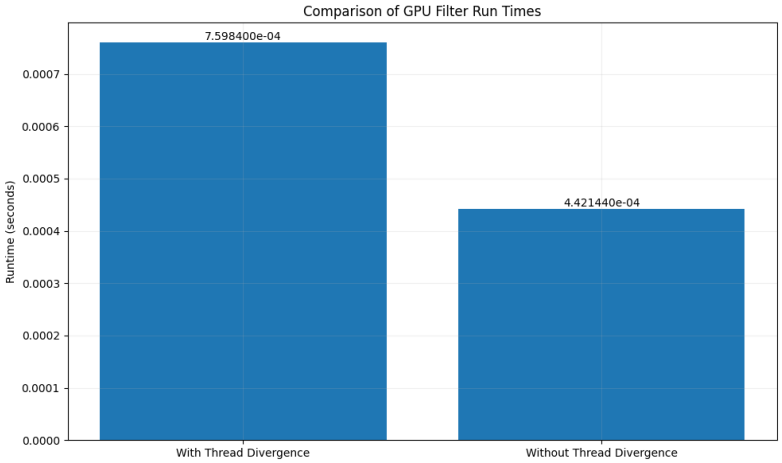


Figuur 1.1: Uitvoertijden voor toenemend aantal threads per block voor inversion functie (tijd in logaritmische schaal)

1.3.3 Thread divergence

We onderzoeken de invloed van thread divergentie door twee implementaties van de Gaussiaanse blur te vergelijken. De eerste implementatie, `REDfilterGPU_WithThreadDivergence` (lijn 140 in appendix A), past de Gaussiaanse blur toe op de rode waarden en inverteert de blauwe en groene waarden binnen dezelfde kernel. Dit leidt tot thread divergentie, omdat threads binnen een warp verschillende instructies uitvoeren (blur vs. inversie), waardoor de threads moeten wachten op elkaar.

Om thread divergentie te verminderen, herstructureren we de afbeeldingsdata met de functie `transformImageData` (lijn 415 in appendix A) zodat de rode, groene en blauwe waarden gegroepeerd zijn (RRR...GGG...BBB). Vervolgens verwerken we deze gereformatteerde data met drie afzonderlijke kernels die gelijktijdig uitgevoerd worden: `redChannelFilter` (lijn 430 in appendix A) voor de Gaussiaanse blur op de rode waarden, `blueChannelInversion` (lijn 445 in appendix A) voor de inversie van de blauwe waarden, en `greenChannelInversion` (lijn 454 in appendix A) voor de inversie van de groene waarden. Door de bewerkingen op te splitsen, verminderen we thread divergentie aanzienlijk, omdat threads binnen een warp nu meer uniforme instructies uitvoeren. Deze vermindering in divergentie resulteert in een duidelijke afname van de runtime. In Figuur 1.2 zien we het verschil in uitvoeringstijd.



Figuur 1.2: Uitvoertijden voor thread divergence tegenover geen thread divergence

1.4 Conclusie

Door de CPU- en GPU-implementaties van image inversie en een Gaussiaanse blur te vergelijken, hebben we aangetoond dat GPU's aanzienlijke prestatieverbeteringen kunnen bieden bij complexere berekeningen per core. Verder hebben we de invloed van de blocksize en striding onderzocht. Vervolgens hebben we de invloed van thread divergentie op de GPU-prestaties aangetoond en hebben we een strategie onderzocht om dit te verminderen door de data te herordenen en aparte kernels te gebruiken voor elk kleurkanaal. De resultaten bevestigen het belang van het overwegen van GPU-architectuur bij het ontwerpen van parallelle algoritmen voor maximale efficiëntie.

A CODE

```
/*
 * Code snippet for importing / exporting image data.
 *
 * To convert an image to a pixel map, run `convert <name>.<extension> <name>.ppm
 *
 */
#include <cstdint>  // Data types
#include <iostream> // File operations
#include <cuda_runtime.h>
#include <stdio.h>
#include <chrono>
#include <iostream>
#include <vector>
#include <random>
#include <ctime>
#include <algorithm>
#include <math.h>

// #define M 512       // Lenna width
// #define N 512       // Lenna height
#define M 960     // VR width
#define N 1280    // VR height
#define C 3       // Colors
#define OFFSET 16 // Header length

uint8_t *get_image_array(void)
{
    /*
     * Get the data of an (RGB) image as a 1D array.
     *
     * Returns: Flattened image array.
     *
     * Noets:
     *  - Images data is flattened per color, column, row.
     *  - The first 3 data elements are the RGB components
     *  - The first 3*M data elements represent the firts row of the image
     *  - For example, r_{0,0}, g_{0,0}, b_{0,0}, ..., b_{0,M}, r_{1,0}, ..., b_{b,M},
    ..., b_{N,M}
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile = fopen("./anna.ppm", "rb");
    if (imageFile == NULL)
    {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

```

```
    // Initialize empty image array
    uint8_t *image_array = (uint8_t *)malloc(M * N * C * sizeof(uint8_t) + OFFSET);

    // Read the image
    fread(image_array, sizeof(uint8_t), M * N * C * sizeof(uint8_t) + OFFSET, imageFile);

    // Close the file
    fclose(imageFile);

    // Move the starting pointer and return the flattened image array
    return image_array + OFFSET;
}

void save_image_array(uint8_t *image_array)
{
    /*
     * Save the data of an (RGB) image as a pixel map.
     *
     * Parameters:
     *  - param1: The data of an (RGB) image as a 1D array
     *
     */
    // Try opening the file
    FILE *imageFile;
    imageFile = fopen("./output_image.ppm", "wb");
    if (imageFile == NULL)
    {
        perror("ERROR: Cannot open output file");
        exit(EXIT_FAILURE);
    }

    // Configure the file
    fprintf(imageFile, "P6\n");          // P6 filetype
    fprintf(imageFile, "%d %d\n", M, N); // dimensions
    fprintf(imageFile, "255\n");         // Max pixel

    // Write the image
    fwrite(image_array, 1, M * N * C, imageFile);

    // Close the file
    fclose(imageFile);
}

struct info
{
    float kernelTime;
    float totalTime;
};

uint8_t *inversionCPU(uint8_t *original, uint8_t *inverted)
{
    for (int i = 0; i < M * N * C; i += 3)
    {
        inverted[i] = 255 - original[i];
    }
    for (int i = 1; i < M * N * C; i += 3)
    {
        inverted[i] = 255 - original[i];
    }
    for (int i = 2; i < M * N * C; i += 3)
    {
```

```
        inverted[i] = 255 - original[i];
    }
    return inverted;
}

uint8_t *REDfilterCPU(uint8_t *original, uint8_t *inverted)
{
    for (int i = 0; i < M * N * C; i += 3)
    {
        inverted[i] = 0.1 * original[i - 2] + 0.25 * original[i - 1] + 0.5 * original[i] +
     0.25 * original[i + 1] + 0.1 * original[i + 2];
    }
    for (int i = 1; i < M * N * C; i += 3)
    {
        inverted[i] = 255 - original[i];
    }
    for (int i = 2; i < M * N * C; i += 3)
    {
        inverted[i] = 255 - original[i];
    }
    return inverted;
}

__global__ void inversionGPU(uint8_t *original, uint8_t *inverted, int size)
{
    for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < size; i += blockDim.x *
    gridDim.x)
    {
        inverted[i] = 255 - original[i];
    }
}

__global__ void REDfilterGPU_WithThreadDivergence(uint8_t *original, uint8_t *inverted,
    int size)
{
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    if (i < size)
    {
        if (i % 3 != 0)
        {
            inverted[i] = 255 - original[i];
        }
        else
        {
            inverted[i] = 0.1 * original[i - 2] + 0.25 * original[i - 1] + 0.5 * original[
    i] + 0.25 * original[i + 1] + 0.1 * original[i + 2];
        }
    }
}



info inversionCPUrun()
{
    // Read the image
    uint8_t *image_array = get_image_array();
    // Allocate host output
    uint8_t *new_image_array = (uint8_t *)malloc(M * N * C);
    // CPU
    // measuring
```

```
    const auto start = std::chrono::steady_clock::now();
    // measuring

    inversionCPU(image_array, new_image_array);

    // measuring
    const auto end = std::chrono::steady_clock::now();
    const std::chrono::duration<double> elapsed_seconds{end - start};
    // measuring

    info info;
    info.kernelTime = elapsed_seconds.count(); // Convert to seconds
    info.totalTime = elapsed_seconds.count();  // Convert to seconds

    return info;
}

info REDfilterCPUrun()
{
    // Read the image
    uint8_t *image_array = get_image_array();
    // Allocate host output
    uint8_t *new_image_array = (uint8_t *)malloc(M * N * C);
    // CPU
    // measuring
    const auto start = std::chrono::steady_clock::now();
    // measuring

    REDfilterCPU(image_array, new_image_array);

    // measuring
    const auto end = std::chrono::steady_clock::now();
    const std::chrono::duration<double> elapsed_seconds{end - start};
    // measuring

    info info;
    info.kernelTime = elapsed_seconds.count(); // Convert to seconds
    info.totalTime = elapsed_seconds.count();  // Convert to seconds

    return info;
}

info inversionGPUrun(int input_blockSize)
{
    // Read the image
    uint8_t *image_array = get_image_array();
    // Allocate host output
    uint8_t *new_image_array = (uint8_t *)malloc(M * N * C);

    // Set up grid and block dimensions
    int blockSize = input_blockSize;
    int numBlocks = (M * N * C + blockSize - 1) / blockSize / 10;

    // Allocate device memory
    uint8_t *d_original, *d_inverted;
    cudaMalloc((void **)&d_original, M * N * C * sizeof(uint8_t));
    cudaMalloc((void **)&d_inverted, M * N * C * sizeof(uint8_t));

    // Measure total time including data transfer
    cudaEvent_t start_cuda_total, stop_cuda_total;
    cudaEventCreate(&start_cuda_total);
```

```
    cudaEventCreate(&stop_cuda_total);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_total);

    // Copy data to device
    cudaMemcpy(d_original, image_array, M * N * C * sizeof(uint8_t),
    cudaMemcpyHostToDevice);

    // Measure kernel execution time separately
    cudaEvent_t start_cuda_kernel, stop_cuda_kernel;
    cudaEventCreate(&start_cuda_kernel);
    cudaEventCreate(&stop_cuda_kernel);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_kernel);

    // Launch kernel
    inversionGPU<<<numBlocks, blockSize>>>(d_original, d_inverted, M * N * C);

    // Measure kernel execution time
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_kernel);
    cudaEventSynchronize(stop_cuda_kernel);
    float ms_kernel;
    cudaEventElapsedTime(&ms_kernel, start_cuda_kernel, stop_cuda_kernel);

    // Copy result back to host
    cudaMemcpy(new_image_array, d_inverted, M * N * C * sizeof(uint8_t),
    cudaMemcpyDeviceToHost);

    // Measure total time including data transfer
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_total);
    cudaEventSynchronize(stop_cuda_total);
    float ms_total;
    cudaEventElapsedTime(&ms_total, start_cuda_total, stop_cuda_total);

    // Save the image
    save_image_array(new_image_array);

    // Free memory
    free(new_image_array);
    cudaFree(d_original);
    cudaFree(d_inverted);

    info info;
    info.kernelTime = ms_kernel / 1000.0; // Convert to seconds
    info.totalTime = ms_total / 1000.0;   // Convert to seconds
    return info;
}

info REDfilterGPUrun_WithThreadDivergence()
{
    // Read the image
    uint8_t *image_array = get_image_array();
    // Allocate host output
    uint8_t *new_image_array = (uint8_t *)malloc(M * N * C);

    // Set up grid and block dimensions
    int blockSize = 256;
    int numBlocks = (M * N * C + blockSize - 1) / blockSize;

```

```
    // Allocate device memory
    uint8_t *d_original, *d_inverted;
    cudaMalloc((void **)&d_original, M * N * C * sizeof(uint8_t));
    cudaMalloc((void **)&d_inverted, M * N * C * sizeof(uint8_t));

    // Measure total time including data transfer
    cudaEvent_t start_cuda_total, stop_cuda_total;
    cudaEventCreate(&start_cuda_total);
    cudaEventCreate(&stop_cuda_total);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_total);

    // Copy data to device
    cudaMemcpy(d_original, image_array, M * N * C * sizeof(uint8_t),
    cudaMemcpyHostToDevice);

    // Measure kernel execution time separately
    cudaEvent_t start_cuda_kernel, stop_cuda_kernel;
    cudaEventCreate(&start_cuda_kernel);
    cudaEventCreate(&stop_cuda_kernel);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_kernel);

    // Launch kernel
    REDfilterGPU_WithThreadDivergence<<<numBlocks, blockSize>>>(d_original, d_inverted, M
    * N * C);

    // Measure kernel execution time
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_kernel);
    cudaEventSynchronize(stop_cuda_kernel);
    float ms_kernel;
    cudaEventElapsedTime(&ms_kernel, start_cuda_kernel, stop_cuda_kernel);

    // Copy result back to host
    cudaMemcpy(new_image_array, d_inverted, M * N * C * sizeof(uint8_t),
    cudaMemcpyDeviceToHost);

    // Measure total time including data transfer
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_total);
    cudaEventSynchronize(stop_cuda_total);
    float ms_total;
    cudaEventElapsedTime(&ms_total, start_cuda_total, stop_cuda_total);

    // Save the image
    save_image_array(new_image_array);

    // Free memory
    free(new_image_array);
    cudaFree(d_original);
    cudaFree(d_inverted);

    info info;
    info.kernelTime = ms_kernel / 1000.0; // Convert to seconds
    info.totalTime = ms_total / 1000.0;   // Convert to seconds
    return info;
}

float getMedian(std::vector<float> &v)
{
```

```
    size_t n = v.size() / 2;
    std::nth_element(v.begin(), v.begin() + n, v.end());
    return v[n];
}

/* int main (void) {
    const int NUM_RUNS = 80;

    // Read the image
    //uint8_t* image_array = get_image_array();

    // Allocate output
    //uint8_t* new_image_array = (uint8_t*)malloc(M*N*C);

    std::vector<float> timesInversionCPU(NUM_RUNS);
    std::vector<float> timesInversionGPU(NUM_RUNS);
    std::vector<float> timesREDfilterCPU(NUM_RUNS);
    std::vector<float> timesREDfilterGPU(NUM_RUNS);

    for (int run = 0; run < NUM_RUNS; run++) {
        cudaDeviceSynchronize();
        timesInversionCPU[run] = inversionCPUrun().kernelTime;
        cudaDeviceSynchronize();
        timesInversionGPU[run] = inversionGPUrun().kernelTime;
        cudaDeviceSynchronize();
        timesREDfilterCPU[run] = REDfilterCPUrun().kernelTime;
        cudaDeviceSynchronize();
        timesREDfilterGPU[run] = REDfilterGPUrun().kernelTime;
        cudaDeviceSynchronize();
    }

    float medianInversionCPU = getMedian(timesInversionCPU);
    float medianInversionGPU = getMedian(timesInversionGPU);
    float medianREDfilterCPU = getMedian(timesREDfilterCPU);
    float medianREDfilterGPU = getMedian(timesREDfilterGPU);

    printf("%.6e\n", medianInversionCPU);
    printf("%.6e\n", medianInversionGPU);
    printf("%.6e\n", medianREDfilterCPU);
    printf("%.6e\n", medianREDfilterGPU);

    //Free allocated memory
    //free(new_image_array);
    //free(image_array);

    return 0;
} */

/* int main(void) {
    const int NUM_RUNS = 10;
    const int MAX_blocksize = 380;


    for (int blocksize = 1; blocksize < MAX_blocksize; blocksize++) {
        std::vector<float> timesInversionGPU(NUM_RUNS);

        for (int run = 0; run < NUM_RUNS; run++) {
            cudaDeviceSynchronize();
            timesInversionGPU[run] = inversionGPUrun(blocksize).kernelTime;
            cudaDeviceSynchronize();
        }
```

```
406
407         float medianInversionGPU = getMedian(timesInversionGPU);
408         if(blocksize < MAX_blocksize-3){
409             printf("%d, %.6e\n", blocksize, medianInversionGPU);
410         }
411     }
412     return 0;
413 } */
414
415 void transformImageData(uint8_t *original, uint8_t *transformed, int size)
416 {
417     for (int i = 0; i < size / 3; ++i)
418     {
419         // Place red pixels first
420         transformed[i] = original[i * 3]; // Red channel
421
422         // Then green pixels
423         transformed[size / 3 + i] = original[i * 3 + 1]; // Green channel
424
425         // Then blue pixels
426         transformed[2 * size / 3 + i] = original[i * 3 + 2]; // Blue channel
427     }
428 }
429
430 __global__ void redChannelFilter(uint8_t* transformed, uint8_t* inverted, int size) {
431     int i = blockIdx.x * blockDim.x + threadIdx.x;
432
433     if (i < size / 3) {
434         int redIndex = i; // Red pixels are in the first third of the array
435
436         // Avoid out-of-bounds memory access by checking the index range
437         if (i > 1 && i < size / 3 - 2) {
438             inverted[redIndex] = 0.1 * transformed[redIndex - 2] + 0.25 * transformed[
    redIndex - 1] + 0.5 * transformed[redIndex] + 0.25 * transformed[redIndex + 1] + 0.1 *
     transformed[redIndex + 2];
439         } else {
440             inverted[redIndex] = transformed[redIndex]; // No filtering for boundary
    pixels
441         }
442     }
443 }
444
445 __global__ void blueChannelInversion(uint8_t* transformed, uint8_t* inverted, int size) {
446     int i = blockIdx.x * blockDim.x + threadIdx.x;
447
448     if (i < size / 3) {
449         int blueIndex = i + 2 * size / 3; // Blue pixels are in the last third of the
    array
450         inverted[blueIndex] = 255 - transformed[blueIndex]; // Inversion of blue channel
451     }
452 }
453
454 __global__ void greenChannelInversion(uint8_t* transformed, uint8_t* inverted, int size) {
455     int i = blockIdx.x * blockDim.x + threadIdx.x;
456
457     if (i < size / 3) {
458         int greenIndex = i + size / 3; // Green pixels are in the second third of the
    array
459         inverted[greenIndex] = 255 - transformed[greenIndex]; // Inversion of green
    channel
460     }
```

```
}

void reverseTransformImageData(uint8_t *inverted, uint8_t *finalImage, int size)
{
    for (int i = 0; i < size / 3; ++i)
    {
        // Place the red pixels back to their original location
        finalImage[i * 3] = inverted[i];

        // Place the green pixels back to their original location
        finalImage[i * 3 + 1] = inverted[size / 3 + i];

        // Place the blue pixels back to their original location
        finalImage[i * 3 + 2] = inverted[2 * size / 3 + i];
    }
}




info REDfilterGPUrun_WithoutThreadDivergence() {
    // Read the image
    uint8_t *image_array = get_image_array();

    // Allocate host output and transformed input
    uint8_t *new_image_array = (uint8_t *)malloc(M * N * C * sizeof(uint8_t));
    uint8_t *transformed_input = (uint8_t *)malloc(M * N * C * sizeof(uint8_t));

    // Transform the input data
    transformImageData(image_array, transformed_input, M * N * C);

    // Set up grid and block dimensions
    int blockSize = 256;
    int numBlocks = (M * N * C + blockSize - 1) / blockSize;

    // Allocate device memory
    uint8_t *d_transformed, *d_inverted;
    cudaMalloc((void **)&d_transformed, M * N * C * sizeof(uint8_t));
    cudaMalloc((void **)&d_inverted, M * N * C * sizeof(uint8_t));

    // Measure total time including data transfer
    cudaEvent_t start_cuda_total, stop_cuda_total;
    cudaEventCreate(&start_cuda_total);
    cudaEventCreate(&stop_cuda_total);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_total);

    // Copy transformed data to device
    cudaMemcpy(d_transformed, transformed_input, M * N * C * sizeof(uint8_t),
    cudaMemcpyHostToDevice);

    // Measure kernel execution time separately
    cudaEvent_t start_cuda_kernel, stop_cuda_kernel;
    cudaEventCreate(&start_cuda_kernel);
    cudaEventCreate(&stop_cuda_kernel);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_kernel);

    // Launch kernels
    redChannelFilter<<<numBlocks, blockSize>>>(d_transformed, d_inverted, M * N * C);
    greenChannelInversion<<<numBlocks, blockSize>>>(d_transformed, d_inverted, M * N * C);
```

```
    blueChannelInversion<<<numBlocks, blockSize>>>(d_transformed, d_inverted, M * N * C);

    // Measure kernel execution time
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_kernel);
    cudaEventSynchronize(stop_cuda_kernel);
    float ms_kernel;
    cudaEventElapsedTime(&ms_kernel, start_cuda_kernel, stop_cuda_kernel);

    // Copy the result back to the host
    cudaMemcpy(transformed_input, d_inverted, M * N * C * sizeof(uint8_t),
    cudaMemcpyDeviceToHost);

    // Reverse transform the data
    reverseTransformImageData(transformed_input, new_image_array, M * N * C);

    // Measure total time including data transfer
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_total);
    cudaEventSynchronize(stop_cuda_total);
    float ms_total;
    cudaEventElapsedTime(&ms_total, start_cuda_total, stop_cuda_total);

    // Save the image
    save_image_array(new_image_array);

    // Free memory
    free(new_image_array);
    free(transformed_input);
    cudaFree(d_transformed);
    cudaFree(d_inverted);

    info info;
    info.kernelTime = ms_kernel / 1000.0; // Convert to seconds
    info.totalTime = ms_total / 1000.0;   // Convert to seconds
    return info;
}

int main(void)
{
    float runZero = REDfilterGPUrun_WithThreadDivergence().kernelTime; // Warm up
    cudaDeviceSynchronize();
    float runOne = REDfilterGPUrun_WithThreadDivergence().kernelTime;
    cudaDeviceSynchronize();

    float runTwo = REDfilterGPUrun_WithoutThreadDivergence().kernelTime; // Warm up
    cudaDeviceSynchronize();
    float runThree = REDfilterGPUrun_WithoutThreadDivergence().kernelTime;
    cudaDeviceSynchronize();

    printf("%.6e\n%.6e", runOne, runThree);
    return 0;
}
```

REFERENTIELIJST

[1] *CUDA: Determining threads per block, blocks per grid*. Stack Overflow. Geraardpleegd op 17 februari 2025. 2010. URL: https://stackoverflow.com/questions/4391162/cuda-determining-threads-per-block-blocks-per-grid.

[2] NVIDIA Corporation. *CUDA C++ Programming Guide*. Geraadpleegd op 13 februari 2025. NVIDIA. verschillend. URL: https://docs.nvidia.com/cuda/cuda-c-programming-guide/.

[3] Alex Minnaar. *Grid-stride loops*. Geraadpleegd op 3 maart 2025. Aug 2019. URL: https://alexminnaar.com/2019/08/02/grid-stride-loops.html.